

Tutorial 8

1. In an Huffman code instance, show that if there is a character with frequency greater than $2/5$, then there is a codeword of length 1. Further, show that if all frequencies are less than $1/3$ then there is no codeword of length 1.

Solution: According to Huffman compression, the two trees that have the least net frequency become the left and right child of a new node, forming a new tree. This process is repeated till there is only one tree left. Recall that in Huffman compression, a tree can have one node (root and leaf) that represents one character, or it can be any binary tree, such that each leaf represents a character and the net frequency of the tree is the sum of frequencies of occurrences of all the leaves (characters) in the text.

1. There exists a frequency more than $2/5$, then there exists a codeword of length 1.

If that character c_0 is not the most frequent character, but it has $f_{c_0} > 2/5$, then there exist two or more characters with frequencies $2/5$ or higher. Let the second be c_1 with $f_{c_1} \geq f_{c_0} > 2/5$. If this is the case, either there are just c_0 and c_1 (they are encoded 0 or 1), or there are more than these two characters, but all the rest have net frequency $1 - f_{c_0} - f_{c_1} < 1/5$. All of those get clubbed together to form one tree, since, until this is the case, the two trees with the least frequency cannot include either of c_0 and c_1 .

After this happens, the 'less frequent' tree T gets clubbed with c_0 first, then the resultant gets clubbed with c_1 [Clubbed = they become children of a new node, forming one tree]. At the end, the c_1 is either the left or right child of the root of the final Huffman tree, and has a codeword of length 1 (codewords '0' or '1')

If c_0 is the most frequent character and we expect it to not have a codeword of length 1, it should be clubbed with another tree sometime before the last step, when there are atleast two trees, both of which do not contain c_0 . Further, all trees except for $\{c_0\}$ and one other tree (that gets clubbed with c_0 whenever it gets clubbed), should have a net frequency greater than or equal to $f_{c_0} > 2/5$. This is possible only when there is one such tree, and there are three trees in all.

Let us examine that case: let T_0 be the tree with highest frequency and T_1 the low frequency tree, that gets clubbed with c_0 . Clearly, $f_{T_0} > 2/5$ from this and the frequency of c_0 , we say that $f_{T_1} < 1/5$. Since T_0 is not a singleton tree, we look at the previous step wherever T_0 was formed from two children, T_2 and T_3 . Without loss of generality, let $f_{T_2} \geq f_{T_3}$.

Then, $2 \cdot f_{T_2} \geq f_{T_2} + f_{T_3} = f_{T_0} > 2/5$ so, before its combination, $f_{T_2} > 1/5 > f_{T_1}$; there existed atleast two trees T_3 and T_1 (more if T_1 was not formed yet), both with net frequency less than that of T_2 . This means that if T_2 was clubbed (it had to be for our case to hold), its combination with T_2 did not follow the Huffman algorithm. This arises a contradiction.

Therefore, if there exists a character of frequency more than $2/5$, then there has to be a character with codeword of length 1.

2. If all frequencies are less than $1/3$, there exists no codeword of length 1.

Let there be a character c_0 that has the luxury of a codeword of length 1. If this were so, at the last iteration of Huffman, we have a singleton tree c_0 (else, it would not have a codeword of length 1), and a tree T_0 . Since $f_{c_0} < 1/3$, $f_{T_0} > 2/3$.

Since this tree T_0 has to contain two or more characters, it must have two subtrees T_2 and T_3 , and those must have been clubbed together in the last step (the only possibility). Without loss of generality, let $f_{T_2} \geq f_{T_3}$. This implies that $2 \cdot f_{T_2} \geq f_{T_2} + f_{T_3} = f_{T_0} > 2/3$

or $f_{T_2} > 1/3 > f_{c_0}$. Then, the two trees with least net frequencies are T_3 and $\{c_0\}$. Those would have been clubbed together, arising in a contradiction.

Therefore, we conclude that there cannot exist a codeword of length 1 if all characters have frequency less than $1/3$.

2. Suppose that there is a source that has three characters 'a', 'b', 'c'. The output of the source cycles in the order of 'a', 'b', 'c' followed by 'a' again, and so on. In other words, if the last output was a 'b', then the next output will either be a 'b' or a 'c'. Each letter is equally probable. Is the Huffman code the best possible encoding? Are there any other possibilities? What would be the pros and cons of this?

Solution: Yes, we can use a location-aware encoding i.e. one wherein the encoding of a character depends on its surroundings, specifically, the character before it. This is a better encoding scheme than Huffman encoding.

For instance, we choose to encode a character with a '1', assuming that it is different from the previous character (there can only be one possible value corresponding to this 1), and a '0' assuming it is same. We can encode the first character using any Huffman tree, say, 1 for 'a', 00 for 'b' and 01 for 'c'. Then, abccaaabbca will be encoded as 11101001011. Also, 010011010110 would correspond to cccabbccabb.

Roughly, this corresponds to one bit per character. Compare that with using on an average $5/3$ bits if we encoded each character individually (and they had roughly same frequency each; true since each letter is equally probable), which would be the case with Huffman encoding. This clearly means that our encoding is better, the big pro. Also, ours does not take a very long time to evaluate (definitely not more than Huffman). Another pro (plus point) is that this is very much scalable to m character systems, say the English alphabet, except, when we know that an 'a' is followed by 'a' or 'b'; 'b' by 'b' or 'c'; ... and 'z' by 'z' or 'a'.

Speaking of cons (downsides) of our encoding, this is not generalizable to arbitrary strings. If 'ac', 'cb' or 'ba' occurred as a substring anywhere (very likely), then our encoding fails (there does not exist an encoding by our scheme). Granted, we know that this is not the case for our source; but we first need to verify that the source is of this form (this can easily be done) so this is not really a con.

Another improvement would be to use something like run-length-encoding algorithm, particularly efficient for strings with long runs like aaaaaaaaaabbbbbbbbbbbbbbcccccccc; but those would be inefficient if there are lot of short runs, particularly, if lot of runs have length 1.

3. Given the following frequencies, compute the Huffman code tree.

Character	a	d	g	j	b	e	h	k	c	f	i	l
Frequency	20	7	8	4	6	25	8	2	6	1	12	1

Solution:

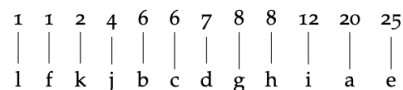


Figure 4: The initial forest for Huffman algorithm

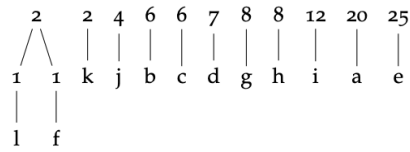


Figure 5: Combining least frequency trees (just 'l' and 'f')

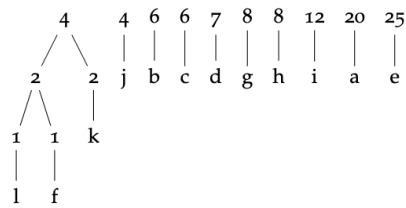


Figure 6: Next iteration, combining trees with least frequency (2 and 2)

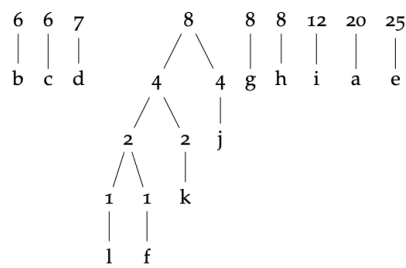
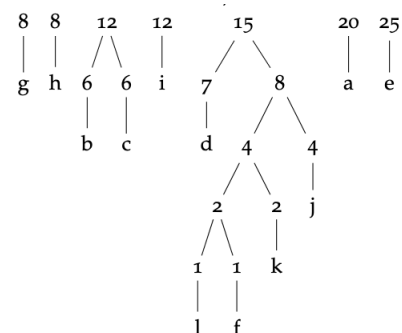
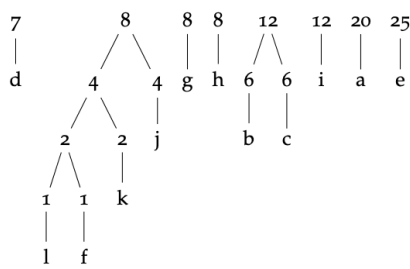
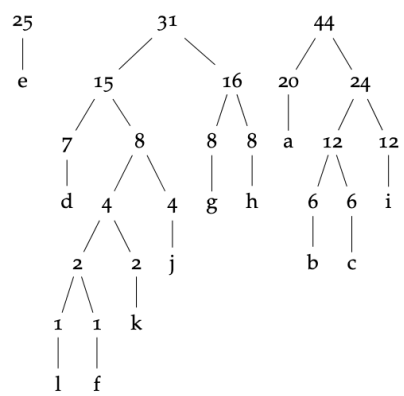
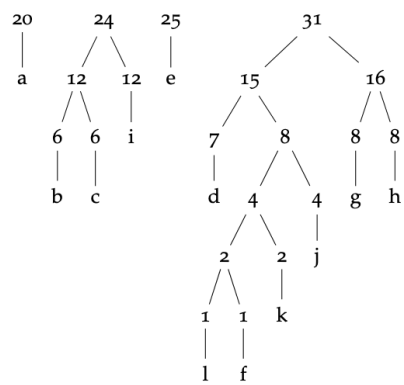
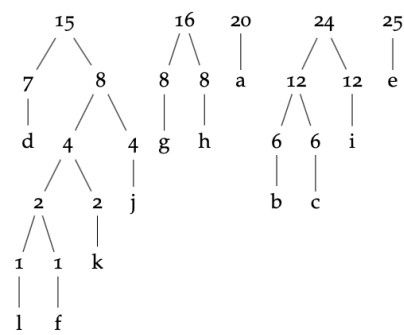
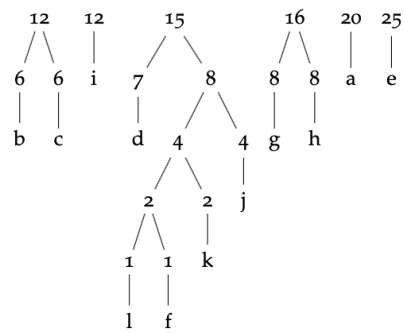


Figure 7: We keep iterating the process as shown below.





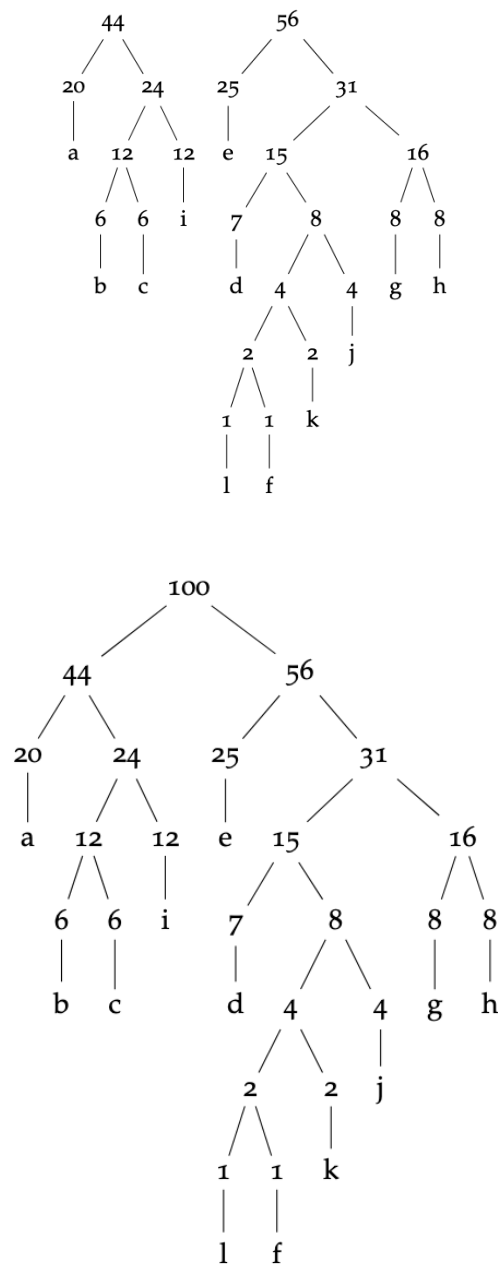


Figure 8: This is the complete Huffman tree.

4. We have seen a “proof of optimality” of the Huffman Coding. Is Huffman Coding really the best lossless text compression method out there? If no, give an example where Huffman Coding costs more than some other coding. Then is Huffman “optimal”? What “optimality” did we talk back there? Can you think why is Huffman not the best then?

Solution: No, Huffman Coding is NOT the best text compression method we have.

There will be 2 cases:- 1) We build a different Huffman tree for every string, 2) We use some universal Huffman tree for all strings.

In case 1, we need to use some bits to store the Huffman tree made every time for an input string. Take “Today_is_Monday” for example. Original string takes $15 \times 8 = 120$

bits (Assuming 8 bits for each ASCII), but Huffman takes *atleast* $49 + 10 \times 8 = 129$ bits (49 for the Huffman string created (You can work this out on your own) and the table (which maps each character of the word to a Huffman code) will contain 10 entries as 10 distinct characters (Also assumed that each entry takes minimum 8 bits)).

In case 2, if there is some pre-processed Huffman table, then take “xxxx”. Since ‘x’ is the least used English alphabet, it’s Huffman code will likely take more than 8 bits.

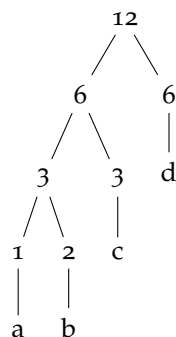
Huffman is “optimal” only when we consider bits used to represent the Huffman string. The proof done in class doesn’t cover the bits that maybe required for making the Huffman table.

Another reason why Huffman isn’t the best is that it assigns *integer*-number of bits to any symbol. Huffman codes are constrained to bit boundaries. In a Shannon information theoretic sense, if we just look at context-free probabilities of symbols, the ideal bit-length of a symbol is $-\log_2(P(x))$ where $P(x)$ is the probability of occurrence of the symbol x .

That number is unlikely to be an integer! The Huffman code’s inefficiency lies in the distance between the information theoretic optimal bit length and the actual integer bit length the Huffman code assigned.

5. What is the largest Huffman code for any character that can be created in a word? Give an example.

Solution: Largest Huffman code for any character can be one less than number of distinct characters in the word. Consider “abbccddddd”:-



See that code length for ‘a’ or ‘b’ is 3.

6. You are given a stick of length l . Your job is to break the stick to create pieces of lengths $l_1, l_2, l_3, \dots, l_m$ (These sum up to l). But the cost of breaking any stick (at any point) into two parts is equal to the length of that stick. What will be the minimum cost to do your job?

Solution: The solution is pretty similar to that of Huffman Coding. We build a tree just like the Huffman tree, just here the weights of the nodes is the length of the sticks instead of frequency of character. Proof of optimality follows from that of Huffman’s.